

2.4 Proceso de Ingeniería de Requisitos

Definición de proceso de IR

"El proceso de ingeniería de requisitos es un conjunto estructurado de actividades para obtener, analizar, documentar, validar y gestionar los requisitos a lo largo del ciclo de vida del sistema." [1]

El proceso de ingeniería de requisitos organiza las tareas necesarias para entender qué necesita el usuario y convertir esas necesidades en requisitos bien definidos. Este proceso no ocurre solo al inicio del proyecto, sino que se mantiene durante todo el desarrollo, permitiendo ajustes cuando sea necesario.

Ejemplo: En la red social para mascotas, el proceso de IR comienza entrevistando a dueños de mascotas, veterinarios y refugios, luego analiza sus necesidades, documenta los requisitos y los valida antes de empezar a programar.

Modelos de proceso: cascada, iterativo, ágil

"Los modelos de proceso definen la secuencia y relaciones entre las actividades de ingeniería de requisitos." [5]

- **Cascada:** Los requisitos se definen completamente al inicio del proyecto, antes de cualquier diseño o implementación. Es útil cuando los requisitos son estables y bien entendidos desde el principio.
- **Iterativo:** Los requisitos se definen y refinan en ciclos repetitivos. Cada iteración agrega más detalle o corrige lo que no funcionaba bien.
- **Ágil:** Los requisitos se capturan como "historias de usuario" y se priorizan en ciclos cortos llamados sprints. Se aceptan cambios frecuentes basados en el feedback continuo del cliente.

Ejemplo: Para la red social de mascotas, un modelo ágil permitiría primero desarrollar el registro de mascotas, luego en otro sprint la funcionalidad de adopción, y después los servicios veterinarios, ajustando cada funcionalidad según lo que los usuarios vayan pidiendo.

Actores del proceso

"Los actores (stakeholders) son personas u organizaciones que tienen interés o influencia en el sistema." [2]

Los actores del proceso de ingeniería de requisitos incluyen a todas las personas que participan o se ven afectadas por el software. Cada actor aporta una perspectiva

diferente y es importante involucrarlos para que los requisitos reflejen las necesidades reales.

Ejemplo en la red social de mascotas:

- Dueños de mascotas (usuarios finales)
- Veterinarios (ofrecen servicios)
- Refugios (publican mascotas en adopción)
- Desarrolladores (construyen el sistema)
- Administradores de la plataforma (gestionan contenido)
- Inversionistas (definen alcance y presupuesto)

Actividades del proceso

"Las actividades principales incluyen elicitación, análisis, especificación, validación y gestión de requisitos." [8]

Las actividades del proceso de ingeniería de requisitos son los pasos concretos que se realizan para transformar las ideas de los usuarios en requisitos listos para implementar. Cada actividad tiene un propósito específico y se relaciona con las demás.

Ejemplo en la red social de mascotas:

- **Elicitación:** Entrevistar a refugios para saber cómo publican mascotas.
- **Análisis:** Detectar que los refugios necesitan filtrar por tamaño y edad.
- **Especificación:** Escribir "El sistema permitirá filtrar mascotas por tamaño y edad".
- **Validación:** Mostrar el requisito a los refugios para confirmar que es correcto.
- **Gestión:** Controlar cambios cuando un refugio pida un nuevo filtro por comportamiento.

2.5 Framework de Pohl y Contexto del Sistema

Actividades core

"Las actividades core (centrales) son aquellas esenciales para construir los requisitos del sistema." [5]

Las actividades core son las tareas fundamentales que siempre deben realizarse en cualquier proyecto de ingeniería de requisitos. Estas actividades garantizan que los requisitos se capturen, negocien y documenten correctamente.

Ejemplo en la red social de mascotas: La elicitación core implica preguntar directamente a los refugios cómo gestionan las adopciones hoy; el análisis core verifica si la función de "publicar mascota" entra en conflicto con la de "solicitar adopción".

Actividades transversales

"Las actividades transversales se aplican a lo largo de todo el proyecto y afectan a todas las actividades core." [5]

Las actividades transversales no se hacen una sola vez, sino que acompañan permanentemente al proyecto. Ayudan a mantener la calidad, el control y la consistencia de los requisitos desde el inicio hasta el final.

Ejemplo en la red social de mascotas: La gestión de cambios (actividad transversal) permite que, si un veterinario pide una nueva función después de tres meses, se evalúe su impacto sin afectar lo que ya está desarrollado.

Artefactos

"Los artefactos son documentos o modelos que se producen durante la ingeniería de requisitos." [5]

Los artefactos son los entregables concretos que se generan en cada actividad. Sirven como evidencia y guía para los desarrolladores y validadores del sistema.

Ejemplo en la red social de mascotas:

- Lista de requisitos funcionales
- Diagrama de contexto del sistema
- Especificación de casos de uso (ej: "Adoptar mascota")
- Matriz de trazabilidad (cada requisito vinculado a su origen)
- Actas de validación con los usuarios

Contexto del sistema

"El contexto del sistema define los límites entre el sistema a desarrollar y su entorno, incluyendo actores externos y sistemas con los que interactúa." [1]

El contexto del sistema ayuda a visualizar qué está dentro del software que se va a construir y qué queda fuera (por ejemplo, sistemas externos o tareas manuales). Esto evita malentendidos sobre lo que el sistema debe o no debe hacer.

Ejemplo en la red social de mascotas: El contexto incluye a dueños, veterinarios y refugios como actores externos. La plataforma se conecta con un servicio de mapas para mostrar veterinarios cercanos, pero el pago de servicios veterinarios lo maneja un sistema externo de pagos (está fuera del límite).

Ocho perspectivas de Pohl

"Las ocho perspectivas del framework de Pohl permiten analizar un sistema desde distintos puntos de vista: objetivos, stakeholders, entorno, restricciones, dominio, comportamiento, interacción y estructura." [5]

El framework de Pohl ayuda a cubrir todos los aspectos importantes de un sistema sin olvidar ninguno. Cada perspectiva responde preguntas diferentes sobre el software.

Ejemplo aplicado a la red social de mascotas (una perspectiva por punto):

1. **Objetivos:** Facilitar adopciones responsables.
2. **Stakeholders:** Dueños, veterinarios, refugios.
3. **Entorno:** Aplicación web/móvil, conexión a internet.
4. **Restricciones:** Solo usuarios registrados publican mascotas.
5. **Dominio:** Las mascotas tienen raza, edad, tamaño.
6. **Comportamiento:** Al publicar una mascota, aparece en la lista de adopción.
7. **Interacción:** El usuario hace clic en "Adoptar" y envía un mensaje al refugio.
8. **Estructura:** Base de datos con tablas de usuarios, mascotas, solicitudes.

2.6 Tipos de Sistemas y Visión del Proyecto

Sistemas de información

"Los sistemas de información procesan, almacenan y recuperan datos para apoyar decisiones o control operativo." [5]

Un sistema de información se enfoca en gestionar grandes volúmenes de datos y facilitar consultas, reportes y transacciones. Es el tipo más común en aplicaciones de gestión y comercio.

Ejemplo: La red social de mascotas es un sistema de información porque almacena perfiles de mascotas, publicaciones de adopción y citas veterinarias.

Sistemas embebidos

"Los sistemas embebidos son componentes de software dentro de un dispositivo de hardware más grande, con recursos limitados." [3]

Estos sistemas controlan dispositivos físicos como electrodomésticos, automóviles o sensores. No son aplicaciones de escritorio o web típicas.

Ejemplo (diferente al contexto principal): Un collar inteligente para mascotas que mide la actividad física y envía los datos a la red social mediante Bluetooth es un sistema embebido.

Sistemas software-intensivos

"Un sistema software-intensivo es aquel donde el software contribuye de manera esencial a la funcionalidad, comportamiento y valor del sistema." [2]

Estos sistemas dependen críticamente del software, aunque incluyan hardware o personas. Si el software falla, el sistema completo deja de funcionar adecuadamente.

Ejemplo: La red social para mascotas es software-intensiva porque la mayor parte de su valor (publicar, buscar, adoptar) viene directamente del software.

Sistemas autoadaptativos

"Los sistemas autoadaptativos modifican su comportamiento en respuesta a cambios en el entorno o en sí mismos." [5]

Estos sistemas aprenden de los datos y ajustan sus reglas automáticamente sin intervención humana constante.

Ejemplo: La red social podría recomendar automáticamente mascotas similares a las que un usuario ha visto antes, mejorando las sugerencias con el tiempo según sus interacciones.

Límite del sistema

"El límite del sistema (system boundary) define qué funciones son responsabilidad del sistema y cuáles corresponden a actores externos u otros sistemas." [1]

Establecer el límite evita que el equipo desarrolle funciones que no corresponden o que dependen de terceros sin control.

Ejemplo: En la red social, el límite incluye publicar mascotas y enviar mensajes entre usuarios. Pero el pago de la adopción (si es necesario) o la verificación veterinaria presencial están fuera del límite del sistema.

Visión del proyecto

Visión del proyecto

"La visión del proyecto describe de forma concisa el propósito, los objetivos, el alcance y los stakeholders clave del sistema, alineando a todo el equipo." [7] [6]

La visión es el "norte" que guía todas las decisiones del proyecto. Debe ser corta, clara y motivadora para todos los involucrados.

Ejemplo de redacción de visión para la red social de mascotas:

"Crear una plataforma accesible y segura que conecte a dueños de mascotas, refugios y veterinarios para facilitar adopciones responsables, servicios de salud animal y comunidades de cuidado, reduciendo el abandono de mascotas en zonas urbanas."

Referencias

- [1] IEEE, ISO/IEC/IEEE 29148:2018 Systems and software engineering — Life cycle processes — Requirements engineering, 2nd ed., 2018.
- [2] IEEE, ISO/IEC/IEEE 15288:2023 Systems and software engineering — System life cycle processes, 2023.
- [3] IEEE, ISO/IEC/IEEE 12207:2017 Systems and software engineering — Software life cycle processes, 2017.
- [4] ISO/IEC 25010:2023 Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model, 2023.
- [5] K. Pohl, Requirements Engineering: Fundamentals, Principles and Techniques, 2025.
- [6] SWEBOK Guide V4.0, Guide to the Software Engineering Body of Knowledge, IEEE Computer Society, 2024.
- [7] Project Management Institute, A Guide to the Project Management Body of Knowledge (PMBOK Guide), 7th ed., 2021.
- [8] IREB, CPRE Foundation Level Syllabus v3.1, International Requirements Engineering Board, 2023.